

ELASTICO

Cycle 4.5

Data Population and End-to-End Verification

2026-08-23

Internal Technical Report

STOP CONDITION: BLOCKER IDENTIFIED

1. Environment

Environment verification confirms which external services are configured and reachable. No actual key values are exposed in this report. The checks below were performed by testing for the presence of non-empty environment variables (for API keys) and HTTP reachability (for Understat).

FOOTBALL_DATA_API_KEY	CONFIGURED
THE_ODDS_API_KEY	CONFIGURED
API_SPORTS_KEY	NOT CONFIGURED (empty value in .env)
NEWSDATA_API_KEY	NOT CONFIGURED (empty value in .env)
DATABASE_URL	CONFIGURED
DIRECT_URL	CONFIGURED
UNDERSTAT (understat.com)	AVAILABLE (HTTP 200, ~2s response)

FAIL: API_SPORTS_KEY is empty. API-Sports sync has never completed successfully.

FAIL: NEWSDATA_API_KEY is empty. News sync cannot function.

PASS: DATABASE_URL and DIRECT_URL are configured and connection verified (670 teams in DB).

PASS: Understat getLeagueData endpoint responds with HTTP 200.

2. Understat Sync

A controlled sync test was performed targeting the Premier League (EPL) for the 2024 season. The test followed the exact code path used by the production cron sync: fetch league teams, resolve to DB entities, then fetch match-level xG data to compute per-game averages.

2.1 League Team Fetch

PASS: getLeagueData/EPL/2024 returned 20 teams with valid IDs and names.

All 20 Premier League teams for the 2024/25 season were returned successfully. Team IDs ranged from 71 (Aston Villa) to 285 (Ipswich). Notable: short_title field is empty for all teams, indicating the new API returns only basic identity data.

2.2 Entity Resolution

PASS: First 3 teams resolved via EXACT name match to API-Sports teams in DB.

The DB contains 23 PL teams (primarily from API-Sports). The entity resolution system correctly matched "Aston Villa", "Everton", and "Bournemouth" via exact case-insensitive name matching. The alias table and normalized matching paths are available for teams with name discrepancies (e.g., "Brighton" to "Brighton and Hove Albion").

2.3 Match-Level xG Fetch (BLOCKER)

FAIL: getTeamData/{id}/2024 returns HTTP 404 for ALL tested team IDs.

FAIL: getMatchData/{id} returns HTTP 404 for ALL tested match IDs.

This is the critical blocker. After the successful getLeagueData call, all subsequent requests to getTeamData and getMatchData return 404 (serving HTML error pages instead of JSON). This was tested with multiple team IDs (71, 83, 87), multiple match IDs (53002), with and without delays between requests, and with different Referer headers. The endpoints consistently return 404.

The `getTeamData` endpoint is the only source of match-level xG data in the Understat integration. Without it, `computeTeamXgFromMatches()` cannot function. The pipeline cannot produce team-level xG per game or match-level `homeXg/awayXg` values.

2.4 Player Stats Endpoint (Partial Success)

WARN: `POST /main/getPlayersStats/` works and returns real player xG data (e.g., Salah xG=27.71).

The player stats POST endpoint is the only Understat endpoint that currently returns xG data. It provides player-level xG, xA, xGChain, and xGBuildup aggregated by season. However, the current pipeline architecture does not use this endpoint for team-level xG computation. Aggregating player xG by team would be a new development task.

Teams Requested	20 (EPL 2024)
Teams Resolved (first 3 tested)	3 (all EXACT)
Teams Unresolved	0 (in tested sample)
Matches Fetched	0 (endpoint 404)
Matches with xG	0 (endpoint 404)
TeamAnalytic Rows Created	0
Errors	getTeamData HTTP 404, getMatchData HTTP 404
Elapsed Time	7,244ms

3. Canonical Entity Resolution

The canonical entity model (`CanonicalTeam` + `SourceIdentity`) exists in the Prisma schema and has corresponding build/link logic in `canonical-entity.ts`. However, the model has never been populated in the production database.

FAIL: `CanonicalTeam` count: 0 (table is empty)

FAIL: `SourceIdentity` count: 0 (table is empty)

The `buildCanonicalEntities()` function is called during the cron sync (step 2.5, after API-Sports team sync). However, because API-Sports sync has never completed successfully (`API_SPORTS_KEY` is empty), no teams from the primary data source exist to canonicalize. The ESPN-sourced teams (which do exist) have never been through the canonicalization process because the sync always times out before reaching the canonical build step.

The sync log analysis confirms this: all 15 recorded syncs are ESPN-only (standings and fixtures). No API-Sports, canonical, or Understat sync operations have ever logged a successful completion. Three syncs recorded "partial" status with the error "approaching Vercel timeout, stopping early".

Canonical Teams Created	0
Source Identities Created	0
Teams Linked	0
Teams Unresolved	N/A (no sync attempted)
Potential Duplicate Identities	0 (no data to duplicate)
Existing Teams in DB	670 (23 PL, mostly ESPN-sourced)

4. Real xG Verification

TeamAnalytic rows are the primary storage location for team-level xG data sourced from Understat. The schema supports truthClass, dataFreshness, sourceTeamId, sourceTeamName, and canonicalTeamId fields for full provenance tracking.

FAIL: Total TeamAnalytic rows: 0

FAIL: Understat-sourced analytics: 0

FAIL: No TeamAnalytic rows exist to verify truthClass, source, or numerical values.

The expected state for genuine Understat data is truthClass=REAL, source=UNDERSTAT, with numerical xgPerGame and xgaPerGame values derived from match-level data. However, since the Understat getTeamData endpoint returns 404, no xG data has ever been computed or persisted. The pipeline has never progressed past the entity resolution step.

5. Match xG Verification

The Match model has homeXg, awayXg, homeXgSource, awayXgSource, homeXgTruthClass, and awayXgTruthClass fields. These are designed to store per-match xG with full provenance. The verification checked all 27 matches in the database.

FAIL: 10 matches have homeXg/awayXg not null, but ALL have xG=0 with null source/truthClass.

FAIL: 0 matches have xG > 0.

FAIL: 0 matches have homeXgSource or awayXgSource set.

The 10 matches with non-null xG values are all from source="unknown" with xG=0. These appear to be demo/simulated data that was seeded with default zero values. They represent future World Cup 2026 matches (Canada vs Mexico, Germany vs USA, etc.) with no real xG data. The remaining 17 matches from source="api-sports" correctly have null xG values.

The presence of xG=0 (rather than null) in the "unknown" source matches is itself a concern. Zero is a valid xG value that could be confused with real data. The expected state for missing data is null, not 0. This suggests the demo data was created with explicit zero values rather than leaving the fields null.

6. API Verification

The /api/teams endpoint includes xG provenance fields when serving from the database. It includes xgPerGame, xgaPerGame, npxGPerGame, xgTruthClass, xgSource, xgFreshness, and xgSyncedAt for each team, sourced from the most recent Understat TeamAnalytic row. This is correctly implemented.

PASS: /api/teams returns provenance fields (xgTruthClass, xgSource, xgFreshness, xgSyncedAt).

The /api/matches/[id] endpoint also includes homeXg, awayXg, homeXgSource, awayXgSource, homeXgTruthClass, and awayXgTruthClass in its response. Additionally, it fetches Understat TeamAnalytic data for both the home and away teams and includes xgPerGame, xgaPerGame, xgTruthClass, xgSource, and xgFreshness on the nested team objects. When falling back to ESPN live data, it correctly labels xgTruthClass as "MISSING" and sets xgPerGame/xgaPerGame to null.

PASS: /api/matches/[id] returns match-level xG provenance (homeXgSource, homeXgTruthClass).

PASS: /api/matches/[id] returns team-level xG analytics from Understat TeamAnalytic.

PASS: ESPN fallback correctly labels xG as MISSING (not 0) when no DB data exists.

7. UI Verification

The UI components were inspected for proper handling of null/missing xG data and correct display of truth class and source information. The compare view and match detail view were the primary focus, as these are the two components identified in the data flow chain.

7.1 Compare View

PASS: Compare view uses "?? null" for xG data (does not transform null to 0).

PASS: Compare view displays truthClass badges (REAL in green, non-REAL in yellow, MISSING suppressed).

PASS: Null xG values display as "--" with a clear "No xG data available" message.

7.2 Match Detail View

FAIL: Match detail uses "?? 0" in 6 locations, silently transforming MISSING xG to 0.

Lines 207-208: `hXg = match.homeXg ?? 0` and `aXg = match.awayXg ?? 0`. These variables are used for the share text and the cumulative xG timeline chart. When xG is null (MISSING), the chart would display a flat line at 0 rather than showing "no data".

Lines 411, 416: `xG {(match.homeXg ?? 0).toFixed(1)}` and `xG {(match.awayXg ?? 0).toFixed(1)}`. These display "xG 0.0" in the match score header when no xG data exists. A user would see "xG 0.0" and might reasonably assume this is real data, when it is actually MISSING.

7.3 Tactical View

FAIL: Tactical view uses `"(xgPerGame ?? 0).toFixed(2)"` in 2 locations (lines 411, 418).

When team xG data is MISSING, the tactical view displays "0.00" instead of "--" or "MISSING". This is the same class of bug as the match detail view.

8. Zero-Fabrication Test

A comprehensive search was performed across all source files for patterns indicating fabricated, hardcoded, or proxy xG data. The following patterns were searched: "shots_on_target * 0.1", "Math.random()", hardcoded xG values (1.5, 1.3, 1.45, 1.10), "default xG", "fake xG", "demo xG", "proxy xG", and "hardcoded xG".

PASS: No "shots_on_target * 0.1" patterns found in any source file.

PASS: No "default xG", "fake xG", "demo xG", or "proxy xG" patterns found.

WARN: 4 files contain `Math.random()`: `predictions.ts`, `prediction-engine.ts`, `sidebar.tsx`, `simulate route`.

The `Math.random()` occurrences in `predictions.ts` and `prediction-engine.ts` are used for Monte Carlo simulation sampling (Poisson random variate generation), which is a legitimate statistical technique. The `sidebar.tsx` and `simulate route` usage should be verified but are likely for UI randomization.

8.1 Lambda Value Origins

The prediction models use hardcoded league-average goal values as lambda parameters. In `predictions.ts`, the ELO model uses `leagueAvgGoals = 1.35` to map ELO expected scores to goal expectations. In the Dixon-Coles model, `baseHome = 1.35` and `baseAway = 1.15` are used as baseline averages. These are standard football analytics constants representing the long-run average goals per team per match, not fabricated xG values. They are model parameters, not data claims.

PASS: Lambda values trace to standard league-average constants (1.35 home, 1.15 away), not fabricated xG.

8.2 UI Null-to-Zero Transforms (Fabrication Risk)

FAIL: 6 instances of "xG ?? 0" in match-detail-view.tsx silently transform MISSING to 0.

FAIL: 2 instances of "xgPerGame ?? 0" in tactical-view.tsx silently transform MISSING to 0.

While these are not "fabrication" in the sense of generating fake data, they are a form of silent data falsification: the UI displays "xG 0.0" when the truthful state is "xG MISSING". A user cannot distinguish between a team that genuinely has 0 xG (extremely rare) and a team for which no xG data has been sourced. This violates the principle that MISSING should be displayed honestly.

9. Freshness Test

Data freshness classification is implemented in canonical-entity.ts via the classifyFreshness() function. It categorizes data into FRESH (less than 24 hours), CURRENT (less than 7 days), SEASON (less than 90 days), and STALE (older than 90 days) based on the syncedAt timestamp.

However, no TeamAnalytic rows exist in the database, so the freshness classification cannot be verified against actual data. The classifyFreshness function itself has correct logic: it uses the current time (Date.now()) minus the syncedAt time to compute age in hours, then applies the thresholds. This would produce accurate freshness labels when data is eventually populated.

WARN: Freshness classification logic is correct but untestable: 0 TeamAnalytic rows exist.

10. Failure Test

The Understat pipeline was tested with an unavailable/error condition (HTTP 404 from getTeamData). The observed behavior was as follows.

PASS: Understat fetch returns null on HTTP 404 (does not throw or crash).

PASS: computeTeamXgFromMatches returns null when no match data is available.

PASS: The cron sync catches Understat errors and logs them without crashing.

WARN: Existing valid data is not overwritten: no valid data exists to overwrite.

The sync code in syncUnderstatAnalytics() checks for null xgData and matchesPlayed === 0, skipping the upsert entirely. This means failed syncs do not corrupt existing data. However, since no data has ever been successfully synced, the "do not overwrite valid data" behavior is untested against a scenario where valid data already exists and a subsequent sync fails.

11. Database Integrity

Database integrity checks verify the schema constraints, uniqueness, and referential integrity of the 22 tables in the Prisma schema. The checks were performed via direct database queries.

Duplicate CanonicalTeams	0 (unique constraint on displayName+leagueCode holds)
Duplicate SourceIdentities	0 (unique constraint on source+externalId holds)
Duplicate Matches (by source+sourceId)	0 (unique constraint holds)
Duplicate Teams (by source+sourceId)	0 (unique constraint holds)
Total Teams	670
Total Matches	27

Total Players	0
Total Standings	192
Total Odds Snapshots	0
Total News Articles	0
Total Sync Logs	15
Total Team Analytics	0

PASS: All unique constraints are enforced. Zero duplicates across all tables.

WARN: 0 Player rows exist (API-Sports player sync has never completed).

WARN: 0 OddsSnapshot rows (The Odds API sync has never been triggered).

WARN: 0 NewsArticle rows (Newsdata.io sync has never been triggered).

12. Build

A full production build was executed to verify there are no TypeScript errors, schema errors, or import errors. The build uses Next.js standalone output mode.

PASS: Build compiled successfully in 22.6 seconds.

PASS: 33 static pages generated in 170.8ms.

PASS: 51 API routes compiled (all dynamic).

PASS: 0 TypeScript errors, 0 schema errors, 0 import errors.

13. Security Verification

A search was performed across all client-side components (src/components/) for exposed API keys, key patterns, or credentials. Only environment variable names (not values) were found in documentation strings within settings-view.tsx and system-monitor-view.tsx. No actual key values reach the frontend.

PASS: No API key values exposed in client-side code.

PASS: Only key NAMES appear in UI documentation text (e.g., "set NVIDIA_API_KEY").

BLOCKER: Real xG Cannot Be Populated

Based on the verification results above, the data flow chain cannot be completed:

```
Understat getLeagueData -- WORKS (team names/IDs only, NO xG) | v Understat getTeamData -- FAILS (HTTP 404, ALL team IDs) Understat getMatchData -- FAILS (HTTP 404, ALL match IDs) | v computeTeamXgFromMatches -- CANNOT COMPUTE (no match data) | v TeamAnalytic persist -- NEVER REACHED (no xG to persist) | v API response -- NO REAL xG DATA TO SERVE | v UI (Compare/Match Detail) -- DISPLAYS NOTHING OR 0 (not real xG)
```

The Understat service changed their site architecture (noted in the codebase as "Aug 2025"). The getTeamData and getMatchData AJAX endpoints that previously provided match-level xG data now return 404. Only the getLeagueData (team identity only) and main/getPlayersStats (player xG) endpoints remain functional.

Without match-level xG data, the pipeline cannot produce team-level xG per game or match-level homeXg/awayXg values. The canonical entity model cannot be verified because the sync that populates it (buildCanonicalEntities) has never executed successfully.

Additionally, two empty API keys (API_SPORTS_KEY and NEWSDATA_API_KEY) block the primary data source from functioning. Without API-Sports team data, entity resolution has no reference database to resolve against, and the canonical entity model cannot be built.

Per the Cycle 4.5 directive: STOP.

Real Understat xG cannot be populated reliably. No data is being fabricated, no proxy xG is being substituted, and no alternative provider is being silently used. The exact blockers are reported above.

Issues Found (Separate from Blocker)

Beyond the primary blocker, the following issues were discovered during verification that should be addressed independently:

UI: match-detail ?? 0	6 locations transform null xG to 0 (fabrication risk)
UI: tactical ?? 0	2 locations transform null xgPerGame to 0 (fabrication risk)
DB: demo matches xG=0	10 "unknown" source matches have xG=0 with null provenance
Keys: API_SPORTS_KEY	Empty in .env, blocks all API-Sports sync
Keys: NEWSDATA_API_KEY	Empty in .env, blocks all News sync
Schema: Match.homeXg	Defaults to null (correct), but demo data has explicit 0